AlohaMini Codebase Documentation

Dual-Arm Mobile Manipulation Robot

Technical Documentation

ManiSkill3 Integration

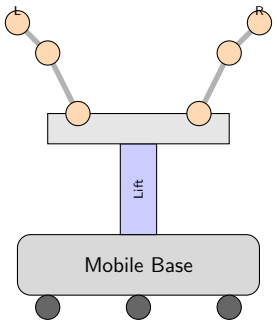
January 14, 2026

Contents

AlohaMini Robot

Dual-Arm Mobile Manipulation Robot

- 16 DOF total configuration
- Omnidirectional mobile base (3 wheels)
- Vertical lift mechanism
- Dual 6-DOF arms
- 5-camera perception system



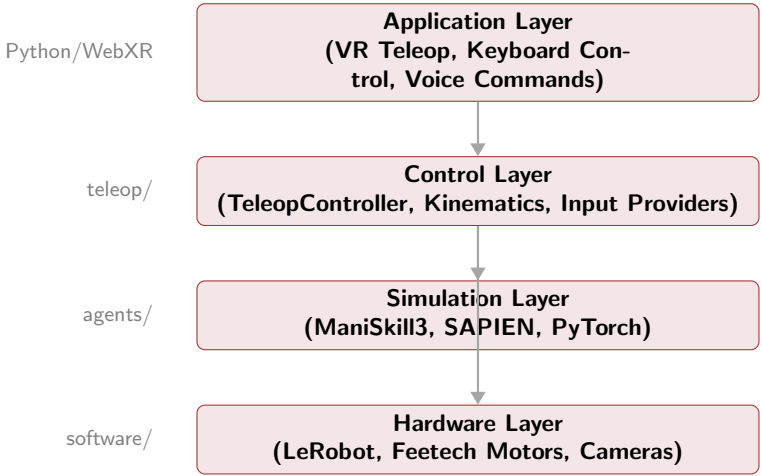
Robot Specifications

Component	DOF	Type
Mobile Base (X, Y, θ)	3	Prismatic/Continuous
Vertical Lift	1	Prismatic
Left Arm	6	Revolute
Right Arm	6	Revolute
Total	**16**	

Action Space (ManiSkill3)

```
[base_x, base_y, base_rot, lift, left_arm(6), right_arm(6)]
```

Software Stack



Key Features

Simulation (ManiSkill3)

- GPU-accelerated physics
- Virtual mobile base
- Grasping detection
- Camera rendering
- ReplicaCAD scenes

Teleoperation

- Keyboard control (XLeRobot-style)
- VR control (WebXR)
- 50 Hz control loop
- IK-based arm control

Physical Robot

- LeRobot integration
- 5-camera perception
- Trajectory recording
- Voice commands

ROS Simulation

- RViz visualization
- Gazebo physics
- URDF models
- ROS1/ROS2 support

Code Metrics

Module	Lines
agents/aloha_mini	∼850
teleop/	∼2,000
kinematics/	∼800
software/ (LeRobot)	∼1,500
Total	∼5,150

Key Classes

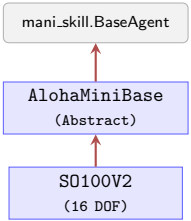
- `AlohaMiniVirtual`
- `TeleopController`
- `AlohaMiniKinematics`
- `ControlGoal`
- `KeyboardController`
- `VRControllerServer`

agents/aloha_mini/ Module

Purpose: Define robot agents for ManiSkill3 simulation

Files:

- `base_agent.py` (208 lines)
- `aloha_mini_so100_v2.py` (351 lines)
- `__init__.py`



base_agent.py - Base Class

```python
class AlohaMiniBaseAgent(BaseAgent):
    """Base agent for variants."""
    BASE_JOINT_NAMES = [
        "root_x_axis_joint",
        "root_y_axis_joint",
        "root_z_rotation_joint",
    ]
    LIFT_JOINT_NAMES = ["vertical_move"]

    @abstractmethod
    def get_left_ee_pose(self): pass

    @abstractmethod
    def get_right_ee_pose(self): pass
```

Common Functionality:

- Joint name definitions
- Controller parameter defaults
- `_create_base_controller()`
- `is_grasping()` (Template Method)
- `is_static()`

Design Pattern

Template Method: `is_grasping()` calls abstract `_check_single_arm_grasping()`

base_agent.py - Controller Helpers

```python
def _create_base_controller(self):
    """Create PDBaseVelController for virtual mobile base."""
    return PDBaseVelControllerConfig(
        self.base_joint_names,
        lower=[-1, -1, -3.14],     # [vx, vy, omega]
        upper=[1, 1, 3.14],
        damping=self.base_damping,         # 1000
        force_limit=self.base_force_limit, # 500
    )

def _create_lift_pos_controller(self):
    """Create position controller for lift mechanism."""
    return PDJointPosControllerConfig(
        self.lift_joint_names,
        stiffness=self.lift_stiffness,     # 2e3
        damping=self.lift_damping,         # 2e2
        force_limit=self.lift_force_limit, # 150
        normalize_action=False,
    )
```

base_agent.py - Grasping Detection

```python
def is_grasping(self, object: Actor, min_force=0.5,
                max_angle=110, arm_id=None):
    """Check if robot is grasping an object."""
    if arm_id is None:
        # Check both arms
        arm1 = self._check_single_arm_grasping(
            object, min_force, max_angle, arm_id=1)
        arm2 = self._check_single_arm_grasping(
            object, min_force, max_angle, arm_id=2)
        return torch.logical_or(arm1, arm2)
    else:
        return self._check_single_arm_grasping(
            object, min_force, max_angle, arm_id)

@abstractmethod
def _check_single_arm_grasping(self, object, ...):
    """Implemented by subclass (different gripper structures)"""
    pass
```

Note

Different gripper structures require different contact force calculations.

aloha_mini_so100_v2.py - SO100 Variant

Configuration:

- `uid = "aloha_mini_so100_v2"`
- Virtual base (prismatic X/Y)
- 16 DOF total
- SO100 arm design

Joint Names:

- `left_shoulder_pan/lift`
- `elbow_flex`, `wrist_flex`
- `wrist_roll`, `gripper`

Controller Parameters:

Parameter	Value
arm_stiffness	3e3
arm_damping	2e2
arm_force_limit	100 N
gripper_stiffness	100

Cameras:

- cam_main (320×240)
- cam_left_wrist (128×128)
- cam_right_wrist (128×128)

Keyframes Configuration

```
keyframes = dict(
    rest=Keyframe(
        qpos=np.array([
            0.0, 0.0, 0.0,  # Base: x, y, rotation
            0.0,            # Lift
            0.0, 0.0, 0.0, 0.0, 0.0, 0.0,  # Left arm
            0.0, 0.0, 0.0, 0.0, 0.0, 0.0,  # Right arm
        ]),
        pose=sapien.Pose(p=[0, 0, 0]),
    ),
    ready=Keyframe(
        qpos=np.array([
            0.0, 0.0, 0.0, 0.05,           # base + lift
            0.0, 0.3, -0.3, 0.0, 0.3, 0.0, # left
            0.0, 0.3, -0.3, 0.0, 0.3, 0.0, # right
        ]),
    ),
)
```

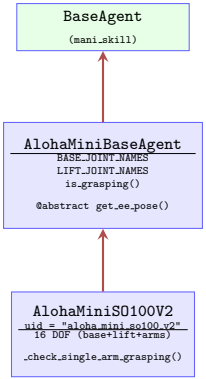
Camera Setup

```python
@property
def _sensor_configs(self):
    q_main = euler_to_quat_xyz(
        math.radians(90), math.radians(90), 0)  # overhead
    q_wrist = euler_to_quat_xyz(
        0, math.radians(45), math.radians(-90)) # 45deg down

    return [
        CameraConfig(uid="cam_main",
            pose=Pose.create_from_pq(p=[0.0, 0.0, 1.45], q=q_main),
            width=320, height=240, fov=1.5, entity_uid="base_link"),
        CameraConfig(uid="cam_left_wrist",
            pose=Pose.create_from_pq(p=[0.0, 0.0, 0.15], q=q_wrist),
            width=128, height=128, fov=1.3, entity_uid="left_link4"),
    ]
```

Grasping Detection

```python
def _check_single_arm_grasping(self, object, min_force=0.5,
                               max_angle=110, arm_id=1):
    """SO100 style: two-finger grasping detection."""
    finger1_link = self.left_finger1_link  # Fixed_Jaw
    finger2_link = self.left_finger2_link  # Moving_Jaw

    l_contact = self.scene.get_pairwise_contact_forces(
        finger1_link, object)
    r_contact = self.scene.get_pairwise_contact_forces(
        finger2_link, object)

    lforce = torch.linalg.norm(l_contact, axis=1)
    rforce = torch.linalg.norm(r_contact, axis=1)

    # Direction to open gripper
    ldirection = finger1_link.pose.to_transformation_matrix()[..., :3, 1]
    rdirection = -finger2_link.pose.to_transformation_matrix()[..., :3, 1]

    # Check force and angle thresholds
    lflag = (lforce >= min_force) & (angle <= max_angle)
    rflag = (rforce >= min_force) & (angle <= max_angle)
    return torch.logical_and(lflag, rflag)
```

Agents Module - Class Diagram

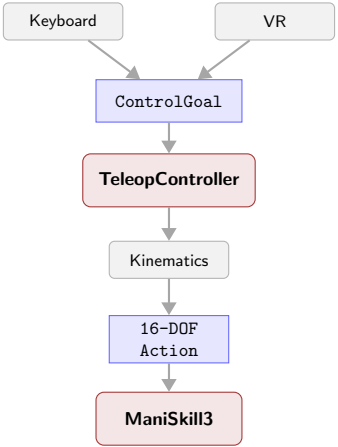


teleop/ Module Overview

Purpose: Real-time teleoperation from keyboard/VR input to ManiSkill3 actions

Key Files:

- `controller.py`
- `config.py`
- `inputs/base.py`
- `inputs/keyboard_controller.py`
- `inputs/vr_ws_server.py`



inputs/base.py - ControlGoal

```python
@dataclass
class ControlGoal:
    """Command message from input provider to controller."""

    # Target position in normalized space
    left_target: Optional[np.ndarray] = None    # [x, y, z]
    right_target: Optional[np.ndarray] = None  # [x, y, z]

    # Direct joint angle targets (radians)
    left_joint_angles: Optional[np.ndarray] = None  # 6 joints
    right_joint_angles: Optional[np.ndarray] = None

    # Gripper state
    left_gripper_closed: bool = False
    right_gripper_closed: bool = False

    # Control mode
    mode: ControlMode = ControlMode.IDLE

    # Mobile base velocity
    base_velocity: Optional[np.ndarray] = None  # [vx, vy, omega]

    # Lift delta
    lift_delta: float = 0.0
```

inputs/base.py - ArmState

```python
@dataclass
class ArmState:
    """Internal state tracking for one arm."""
    ee_pos: np.ndarray          # [y, z] in arm plane
    joint_angles: np.ndarray    # 6 joints (radians)
    pitch: float = 0.0          # Elbow angle sum
    gripper_closed: bool = False
    target_pos: Optional[np.ndarray] = None
```

Coordinate System

- ee_y: Forward direction (usually negative at home)
- ee_z: Height (positive = up)

inputs/base.py - ControlMode

```python
class ControlMode(Enum):
    """Control mode for teleoperation."""

    IDLE = "idle"           # No active control
    POSITION = "position"   # Position control (IK-based)
    VELOCITY = "velocity"   # Velocity control (direct joint)
```

Mode	Input	Processing
IDLE	None	Hold current position
POSITION	Target [x,y,z]	IK solver computes joints
VELOCITY	Joint deltas	Direct joint angle update

controller.py - TeleopController

Responsibilities:

- Process keyboard/VR input
- Compute IK for both arms
- Generate 16-DOF action array
- Track arm states
- Handle mobile base control

Key Methods:

- `process_goal()`
- `compute_action()`
- `update_arm_state()`
- `get_action()`

State Variables:

- `left_arm_state: ArmState`
- `right_arm_state: ArmState`
- `current_lift: float`
- `kinematics: AlohaMiniKinematics`

Configuration:

- Control frequency: 50 Hz
- Position step: 4mm
- Angle step: 1°

controller.py - process_goal()

```python
def process_goal(self, goal: ControlGoal):
    """Process a control goal and update internal state."""

    # Handle mobile base velocity
    if goal.base_velocity is not None:
        self.base_velocity = goal.base_velocity

    # Handle lift
    if goal.lift_delta != 0:
        self.current_lift += goal.lift_delta
        self.current_lift = np.clip(
            self.current_lift, 0.0, 0.6)  # 0-60cm

    # Handle left arm
    if goal.left_target is not None:
        self._update_arm_from_target(
            self.left_arm_state, goal.left_target)
    elif goal.left_joint_angles is not None:
        self.left_arm_state.joint_angles = goal.left_joint_angles

    # Handle gripper
    self.left_arm_state.gripper_closed = goal.left_gripper_closed

    # Similar for right arm...
```

controller.py - compute_action()

```python
def compute_action(self) -> np.ndarray:
    """Generate 16-DOF action array for ManiSkill3."""
    action = np.zeros(16)
    action[0:3] = self.base_velocity      # [vx, vy, omega]
    action[3] = self.current_lift         # Lift position
    action[4:10] = self.left_arm_state.joint_angles    # Left arm
    action[10:16] = self.right_arm_state.joint_angles # Right arm
    return action
```

Action Space Layout

`[vx, vy, omega, lift, L1..L6, R1..R6]`

inputs/keyboard_controller.py

Left Arm Keys:

Keys	Action
Q/A	Joint 1 (+/-)
W/S	Joint 2 (+/-)
E/D	Joint 3 (+/-)
R/F	Joint 4 (+/-)
T/G	Joint 5 (+/-)
Y/H	Gripper

Right Arm Keys:

Keys	Action
U/J	Joint 1 (+/-)
I/K	Joint 2 (+/-)
O/L	Joint 3 (+/-)
P/;	Joint 4 (+/-)
[/'	Joint 5 (+/-)
]/\	Gripper

Mobile Base: Numpad 8/5 (forward/back), 4/6 (left/right), 7/9 (rotate)
Lift: PageUp/PageDown

keyboard_controller.py - Key Processing

```python
def process_input(self, keys) -> ControlGoal:
    """Process pygame key states to ControlGoal."""

    goal = ControlGoal(mode=ControlMode.VELOCITY)
    joint_deltas = np.zeros(6)

    # Left arm joint control
    if keys[pygame.K_q]: joint_deltas[0] += self.angle_step
    if keys[pygame.K_a]: joint_deltas[0] -= self.angle_step
    if keys[pygame.K_w]: joint_deltas[1] += self.angle_step
    if keys[pygame.K_s]: joint_deltas[1] -= self.angle_step
    # ... more joints

    # Apply deltas to current state
    goal.left_joint_angles = self.current_left + joint_deltas

    # Mobile base
    base_vel = np.zeros(3)
    if keys[pygame.K_KP8]: base_vel[0] = 0.3  # forward
    if keys[pygame.K_KP5]: base_vel[0] = -0.3 # backward
    goal.base_velocity = base_vel

    return goal
```

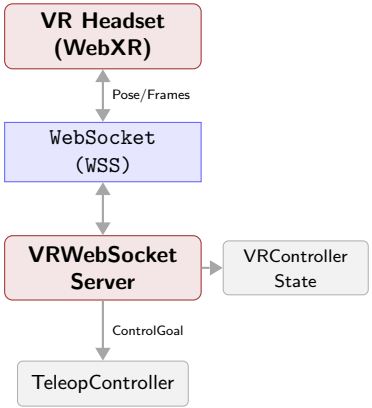
inputs/vr_ws_server.py - VR Interface

Features:

- WebXR controller input
- HTTPS + WebSocket Secure
- Real-time camera streaming
- Pose-to-IK conversion

Protocol:

1. Client connects via WSS
2. Sends controller poses (JSON)
3. Server converts to ControlGoal
4. Streams camera frames back



vr_ws_server.py - Message Format

```
# Incoming VR controller message
{
    "type": "controller_update",
    "left": {
        "position": [x, y, z],       # World coordinates
        "rotation": [qx, qy, qz, qw],
        "buttons": {
            "trigger": 0.8,          # Gripper control
            "grip": true
        }
    },
    "right": { ... },
    "head": {
        "position": [x, y, z],
        "rotation": [qx, qy, qz, qw]
    }
}
```

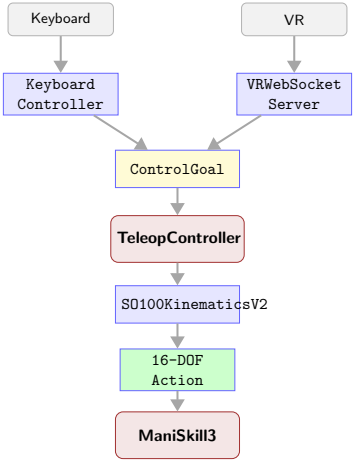
Scaling

VR positions are scaled by `vr_position_scale_xy = 0.7` before IK

config.py - TeleopConfig

```python
@dataclass
class TeleopConfig:
    """Teleoperation configuration."""

    # Network settings
    host_ip: str = "0.0.0.0"
    ws_port: int = 8765
    https_port: int = 8443

    # SSL certificates
    cert_file: str = "cert.pem"
    key_file: str = "key.pem"

    # Control parameters
    control_freq: int = 50        # Hz
    pos_step: float = 0.004       # 4mm per step
    angle_step: float = 1.0       # 1 degree per step

    # IK parameters (from URDF)
    l1: float = 0.116             # Upper arm length
    l2: float = 0.134             # Lower arm length

    # VR scaling
    vr_position_scale_xy: float = 0.7
    vr_position_scale_z: float = 1.0
```

Teleoperation Data Flow



Demo Scripts

Script	Description
demo_teleop.py	Keyboard teleoperation (pygame)
demo_vr_teleop_stream.py	VR teleoperation (WebXR)

Usage:
Keyboard: `python demo_teleop.py`
VR: `python demo_vr_teleop_stream.py`

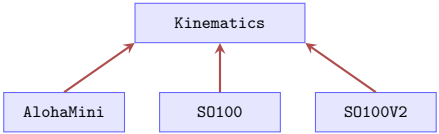
kinematics/ Module Overview

Purpose: IK/FK solvers for arm control

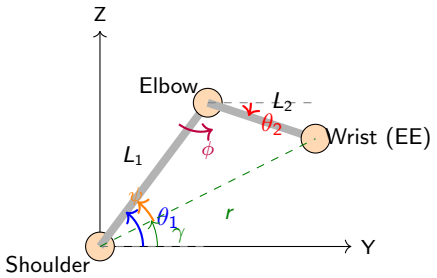
Files:

- `aloha_mini_kinematics.py`
- `so100_kinematics*.py`

Features: 2-link planar IK, Law of Cosines, workspace clamping



2-Link Planar IK - Theory



Law of Cosines:

Distance: $r = \sqrt{y^2 + z^2}$

Interior elbow angle (ϕ):

$$\cos\phi = \frac{L_1^2 + L_2^2 - r^2}{2L_1L_2}$$

Shoulder offset (ψ):

$$\cos\psi = \frac{r^2 + L_1^2 - L_2^2}{2rL_1}$$

Joint angles (elbow-up):

$$\theta_1 = \gamma - \psi$$

AlohaMiniKinematics - URDF Parameters

```
# URDF Joint Chain (Left Arm):
# left_joint2: xyz=[-0.017, -0.031, 0.054]
# left_joint3: xyz=[0.003, 0.169, 0.043]
# left_joint4: xyz=[0.000, -0.201, 0.005]

class AlohaMiniKinematics:
    def __init__(self):
        # 1.5x extended arm lengths
        self.y3 = 0.16857   # joint3 Y offset
        self.z3 = 0.043244  # joint3 Z offset
        self.y4 = -0.20122  # joint4 Y (NEGATIVE!)
        self.z4 = 0.00544   # joint4 Z offset

        # Link lengths from URDF (Pythagorean)
        self.l1 = sqrt(y3^2 + z3^2)  # ~0.174m
        self.l2 = sqrt(y4^2 + z4^2)  # ~0.201m

        # Geometry angles (measured from +Y axis)
        self.alpha1 = atan2(z3, y3)  # ~14.4 deg
        self.alpha2 = atan2(z4, y4)  # ~178.5 deg
```

Home Position

At $j_2 = 0$, $j_3 = 0$: Wrist at $y = -0.022m$, $z = 0.032m$ (Verified: 0.0mm error)

AlohaMiniKinematics - Forward Kinematics

```python
def forward_kinematics(self, j2_deg: float, j3_deg: float) -> Tuple[float, float]:
    """Compute wrist position at joint4."""
    j2 = math.radians(j2_deg)
    j3 = math.radians(j3_deg)

    # Absolute angle of link1 (upper arm) in Y-Z plane
    # When j2=0, link1 points at angle alpha1 from +Y axis
    theta1 = j2 + self.alpha1

    # Absolute angle of link2 (forearm) in Y-Z plane
    # j2 rotates the whole arm, j3 rotates link2 relative to link1
    # alpha2 accounts for the backward-pointing geometry (~178.45 deg)
    theta2 = j2 + j3 + self.alpha2

    # FK: position of j4 relative to j2 in Y-Z plane
    y = self.l1 * math.cos(theta1) + self.l2 * math.cos(theta2)
    z = self.l1 * math.sin(theta1) + self.l2 * math.sin(theta2)

    return y, z
```

Key Insight

The $\alpha_2 \approx 178.45$ offset encodes that the forearm points **backward** in link3's frame. This is why the arm is folded (not extended) at home position.

AlohaMiniKinematics - Inverse Kinematics

```python
def inverse_kinematics(self, y_target: float, z_target: float) -> Tuple[float, float]:
    # Distance from shoulder to target
    r = math.sqrt(y_target**2 + z_target**2)

    # Clamp to workspace
    r_max = self.l1 + self.l2
    r_min = abs(self.l1 - self.l2)
    # ... clamping logic ...

    # Angle to target from +Y axis
    gamma = math.atan2(z_target, y_target)

    # Law of cosines for elbow
    cos_alpha = (r**2 + self.l1**2 - self.l2**2) / (2 * self.l1 * r)
    alpha = math.acos(cos_alpha)

    # Two solutions: theta1 = gamma +/- alpha
    theta1_a = gamma - alpha  # elbow-up
    theta1_b = gamma + alpha  # elbow-down

    # Interior elbow angle
    cos_phi = (self.l1**2 + self.l2**2 - r**2) / (2 * self.l1 * self.l2)
    phi = math.acos(cos_phi)

    # theta2 = theta1 + (pi - phi)  [exterior angle relationship]
    # ... solve and verify with FK ...
```

SO100Kinematics - XLeRobot Style

```python
class SO100Kinematics:
    """XLeRobot-style kinematics."""

    def __init__(self):
        # Link lengths from URDF
        self.l1 = 0.1159  # shoulder-elbow
        self.l2 = 0.1350  # elbow-wrist

        # URDF geometry offsets
        self.theta1_offset = atan2(0.028, 0.11257)
        self.theta2_offset = (
            atan2(0.0052, 0.1349) +
            self.theta1_offset
        )

        # Initial EE position
        self.initial_ee_x = 0.162
        self.initial_ee_y = 0.118
```

URDF Joint Offsets:

Joint	Offset (XYZ)
shoulder_pan	(0, -0.045, 0.017)
shoulder_lift	(0, 0.103, 0.031)
elbow_flex	(0, 0.113, 0.028)
wrist_flex	(0, 0.005, 0.135)

Joint Limits:

- shoulder_pan: ± 114
- shoulder_lift: -6 to 198
- elbow_flex: -11 to 180
- wrist_flex: ± 103

SO100Kinematics - IK with Pitch Compensation

```python
def inverse_kinematics(self, x: float, y: float, pitch: float = 0.0):
    r = math.sqrt(x**2 + y**2)

    # Workspace clamping
    r_max = self.l1 + self.l2
    r_min = abs(self.l1 - self.l2)
    # ... scale if out of bounds ...

    # Law of cosines (NOTE: negative sign is critical!)
    cos_theta2 = -(r**2 - self.l1**2 - self.l2**2) / (2 * self.l1 * self.l2)
    theta2 = math.pi - math.acos(cos_theta2)

    # Shoulder angle using geometric approach
    beta = math.atan2(y, x)
    gamma = math.atan2(self.l2 * sin(theta2), self.l1 + self.l2 * cos(theta2))
    theta1 = beta + gamma

    # Convert to URDF joint angles (add offsets)
    joint2 = theta1 + self.theta1_offset  # shoulder_lift
    joint3 = theta2 + self.theta2_offset  # elbow_flex

    # Wrist flex for pitch compensation
    wrist_flex = joint2 - joint3 + pitch
    return joint2, joint3, wrist_flex
```

SO100KinematicsV2 - V2 URDF Structure

V2 URDF Differences:

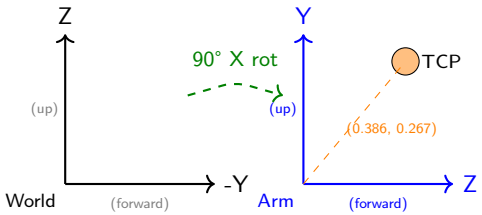
- shoulder_pan: 90° X rotation
- Transforms coordinates:
  - Arm local Y → World Z (up)
  - Arm local Z → World -Y (forward)
- Uses TCP position directly

User Coordinates:

- `ee_x` = forward distance
- `ee_y` = height (up)

TCP at Home:

- $x = 0.386m$ (forward)
- $y = 0.267m$ (up)



Offset Parameters

`offset_x = 0.223m`

`offset_y = 0.150m`

(Accounts for base, shoulder, gripper)

SO100KinematicsV2 - Consistent FK/IK

```python
def forward_kinematics(self, j2: float, j3: float) -> Tuple[float, float]:
    # Absolute angles in arm's Y-Z plane (from +Y toward +Z)
    theta1 = j2 + self.alpha1  # Link 1 angle
    theta2 = j2 + j3 + self.alpha2  # Link 2 angle

    # Wrist position (Y=up, Z=forward in arm frame)
    wrist_y = self.l1 * cos(theta1) + self.l2 * cos(theta2)
    wrist_z = self.l1 * sin(theta1) + self.l2 * sin(theta2)

    # Transform to user coordinates
    ee_x = wrist_z + self.offset_x  # forward
    ee_y = wrist_y + self.offset_y  # up
    return ee_x, ee_y

def inverse_kinematics(self, x: float, y: float, pitch: float = 0.0):
    # Transform TCP to wrist position
    wrist_z = x - self.offset_x
    wrist_y = y - self.offset_y
    # ... law of cosines solution (same as AlohaMiniKinematics) ...
    # Convert: j2 = theta1 - alpha1, j3 = theta2 - theta1 + alpha1 - alpha2
```

Kinematics Verification Results

AlohaMiniKinematics:

Test	Result
FK at home	0.0mm error
IK round-trip	0.0mm error
Joint tracking	0.0° error

SO100KinematicsV2:

Test	Result
FK at home	0.0mm error
IK at home	$\sim$0° joint error
Position round-trip	$<$1mm error

Warning in Code

"DO NOT MODIFY THIS IK/FK CODE"
"Verified working perfectly: 2024-12-31"

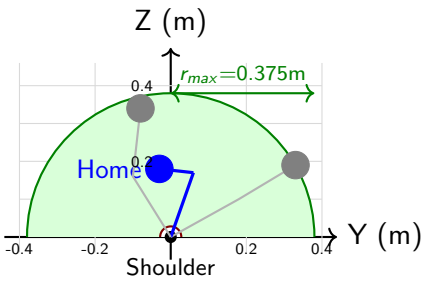
Test Cases:

- Joint angles: 10 test cases
- Position targets: 8 test cases
- All pass with $<$1mm error

Key Insight

Mathematical consistency between FK and IK is critical. Both must use the same geometry angles (α_1, α_2) and coordinate transformations.

Workspace Visualization



Workspace Definition:

- Shoulder at origin (0, 0)
- Arm operates in Y-Z plane
- Green area = reachable zone

Boundaries:

- $r_{max} = L_1 + L_2 = 0.375\text{m}$
- $r_{min} = |L_1 - L_2| = 0.027\text{m}$

Arm Poses:

- Blue: Home (folded)
- Gray: Sample configs

Workspace Parameters

AlohaMiniKinematics:

Parameter	Value
L_1 (upper arm)	0.174m
L_2 (forearm)	0.201m
r_{min}	0.027m
r_{max}	0.375m

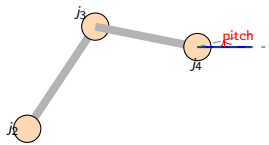
SO100Kinematics:

Parameter	Value
L_1 (upper arm)	0.116m
L_2 (forearm)	0.135m
r_{min}	0.019m
r_{max}	0.251m

Workspace Clamping

IK clamps targets to $[r_{min} \times 1.01, r_{max} \times 0.99]$ to avoid singularities at workspace boundaries.

Wrist Flex / Pitch Compensation



Pitch Formula:

$$pitch_{EE} = j_2 + j_3 + j_4$$

For level gripper ($pitch = 0$):

$$j_4 = -j_2 - j_3$$

Implementation Variants:

AlohaMiniKinematics:

- `j4 = pitch - j2 - j3`

SO100Kinematics:

- `wrist = j2 - j3 + pitch`

SO100KinematicsV2:

- `wrist = pitch - (j2 + j3)`

Key Insight

Setting `pitch=0` keeps gripper level regardless of arm pose.

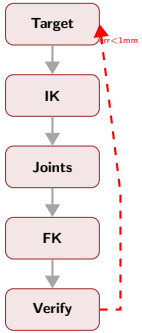
Kinematics Module Summary

Key Design Decisions:

1. 2-link planar IK (Y-Z plane)
2. Law of cosines solution
3. URDF geometry offsets
4. Workspace clamping
5. Pitch compensation

Important Constants:

- Link lengths from URDF
- Geometry angles (α_1, α_2)
- TCP offsets (V2 only)



Critical

FK and IK must be mathematically consistent!

software/ Module Overview

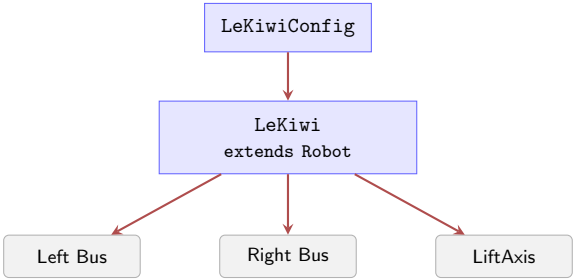
Purpose: Physical robot control via LeRobot framework

Key Files:

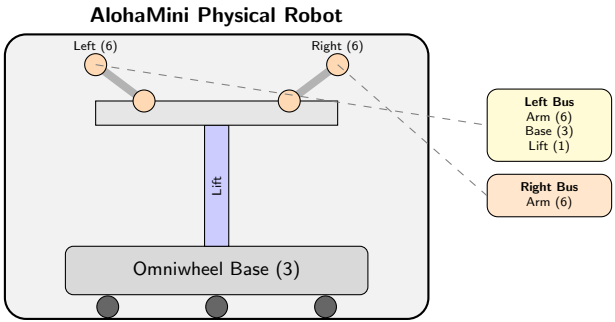
- `lekiwi.py` (642 lines)
- `config_lekiwi.py` (114 lines)
- `lift_axis.py` (217 lines)

Dependencies:

- LeRobot framework
- Feetech motors (STS3215)
- OpenCV cameras



AlohaMini Hardware Architecture



LeKiwi - Motor Configuration

```
self.left_bus = FeetechMotorsBus(
    port=self.config.left_port,
    motors={
        # Left arm joints (6 DOF)
        "arm_left_shoulder_pan": Motor(1, "sts3215", norm_mode),
        "arm_left_shoulder_lift": Motor(2, "sts3215", norm_mode),
        "arm_left_elbow_flex": Motor(3, "sts3215", norm_mode),
        "arm_left_wrist_flex": Motor(4, "sts3215", norm_mode),
        "arm_left_wrist_roll": Motor(5, "sts3215", norm_mode),
        "arm_left_gripper": Motor(6, "sts3215", RANGE_0_100),
        # Mobile base (3 wheels)
        "base_left_wheel": Motor(8, "sts3215", RANGE_M100_100),
        "base_back_wheel": Motor(9, "sts3215", RANGE_M100_100),
        "base_right_wheel": Motor(10, "sts3215", RANGE_M100_100),
        # Lift mechanism
        "lift_axis": Motor(11, "sts3215", DEGREES),
    },
)
```

LeKiwi - State/Action Features

State Features (16 DOF):

- `arm_left_*.pos` (6)
- `arm_right_*.pos` (6)
- `x.vel, y.vel, theta.vel` (3)
- `lift_axis.height_mm` (1)

Camera Features (5):

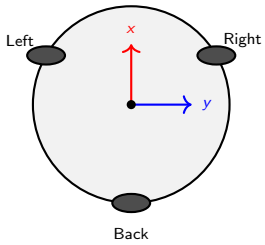
- head_top, head_back, head_front
- wrist_left, wrist_right
- Resolution: 640×480

Component	DOF
Left Arm	6
Right Arm	6
Base Velocity	3
Lift Height	1
Total	**16**

Action = State

Action features mirror state features for teleoperation recording.

Omniwheel Base Kinematics



Inverse Kinematics:

Wheel angles: $\phi_i = [150, 270, 30]$

Kinematic matrix:

$$M = \begin{bmatrix} \cos\phi_1 & \sin\phi_1 & R \\ \cos\phi_2 & \sin\phi_2 & R \\ \cos\phi_3 & \sin\phi_3 & R \end{bmatrix}$$

Body to wheel: $\vec{\omega} = \frac{M \cdot \vec{v}}{r}$

Parameters:

- $r_{wheel} = 0.05m$
- $R_{base} = 0.125m$

Body to Wheel Conversion

```python
def _body_to_wheel_raw(self, x, y, theta, wheel_radius=0.05,
                       base_radius=0.125, max_raw=3000):
    """Convert body velocities (x, y, theta) to wheel raw commands."""
    theta_rad = theta * (np.pi / 180.0)
    velocity_vector = np.array([-x, -y, theta_rad])

    # Wheel mounting angles with -90 deg offset
    angles = np.radians(np.array([240, 0, 120]) - 90)
    m = np.array([[np.cos(a), np.sin(a), base_radius] for a in angles])

    # Compute wheel speeds
    wheel_linear_speeds = m.dot(velocity_vector)
    wheel_angular_speeds = wheel_linear_speeds / wheel_radius
    wheel_degps = wheel_angular_speeds * (180.0 / np.pi)

    # Scale if exceeds max_raw ...
    wheel_raw = [self._degps_to_raw(deg) for deg in wheel_degps]
    return {"base_left_wheel": wheel_raw[0], ...}
```

LiftAxis - Configuration

```python
@dataclass
class LiftAxisConfig:
    enabled: bool = True
    name: str = "lift_axis"
    motor_id: int = 11

    # Lead screw parameters
    lead_mm_per_rev: float = 84
    soft_min_mm: float = 0.0
    soft_max_mm: float = 600

    # Homing parameters
    home_down_speed: int = 1300
    home_stall_current_ma: int = 150

    # Velocity control
    kp_vel: float = 300
    v_max: int = 1300
    dir_sign: int = -1
```

Key Features:

- Multi-turn tracking
- Velocity closed-loop
- Automatic homing
- Soft limits

Position Calc:

$$h_{mm} = (\theta - \theta_0) \times \frac{lead}{360}$$

LiftAxis - Homing Procedure

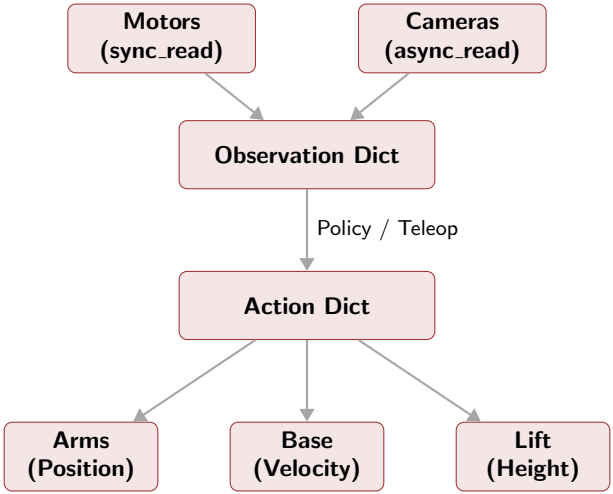
Homing Algorithm:

1. Drive down at `home_down_speed`
2. Monitor current (6.5x scale)
3. Detect stall (>1500mA)
4. Set current position as z=0

Parameters:

- Timeout: 30 seconds
- Poll rate: 50ms
- Stall threshold: 2 consecutive
- Current limit: 1500 mA

LeKiwi - Observation & Action Flow



LeKiwi - Safety Features

1. Overcurrent Protection

- Default limit: 2000 mA
- Scale factor: 6.5 (STS3215)
- Emergency stop on exceed
- Auto-disconnect

2. Position Safety

- `max_relative_target`
- Caps relative motion magnitude
- Prevents sudden jumps

3. Lift Soft Limits

- `soft_min_mm` = 0
- `soft_max_mm` = 600
- Velocity blocked at limits

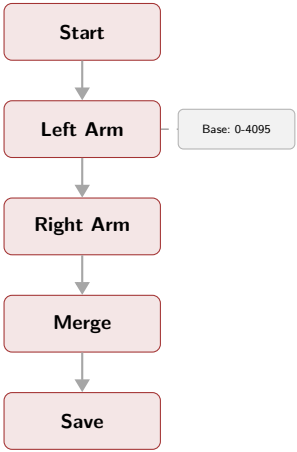
4. Watchdog (Client Mode)

- Timeout: 1500 ms
- Stops robot if no command
- Network disconnection safety

Calibration Process

Dual-Bus Calibration:

1. Disable torque on arm motors
2. Move LEFT arm to middle
3. Record half-turn homing offset
4. Move through full ROM
5. Record min/max positions
6. Repeat for RIGHT arm
7. Merge and save calibration



LeKiwiConfig

```python
@RobotConfig.register_subclass("lekiwi")
@dataclass
class LeKiwiConfig(RobotConfig):
    left_port: str = "/dev/am_arm_follower_left"
    right_port: str = "/dev/am_arm_follower_right"
    disable_torque_on_disconnect: bool = True
    max_relative_target: int | None = None
    cameras: dict[str, CameraConfig] = field(default_factory=...)
    use_degrees: bool = False
```

Camera Device Paths:

- /dev/am_camera_head_top
- /dev/am_camera_head_back, head_front
- /dev/am_camera_wrist_left, wrist_right

LeRobot Module Summary

Key Classes:

- `LeKiwi` - Robot interface
- `LeKiwiConfig` - Configuration
- `LiftAxis` - Z-axis controller

Hardware:

- Feetech STS3215 servos
- FeetechMotorsBus
- 5 OpenCV cameras

Control Modes:

Component	Mode
Arms	Position
Base wheels	Velocity
Lift axis	Velocity

Control Loop: 30-50 Hz

simulation/ Module Overview

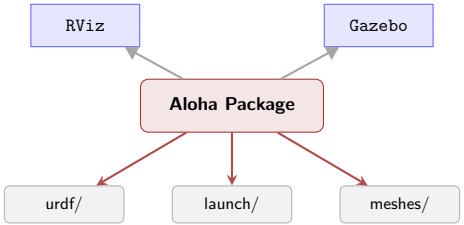
Purpose: ROS visualization & physics

Structure:

- `src/Aloha/` - ROS package
- `urdf/` - Robot model
- `meshes/` - STL files
- `launch/` - Launch files

Support:

- ROS1 Noetic / ROS2 Humble
- RViz / Gazebo

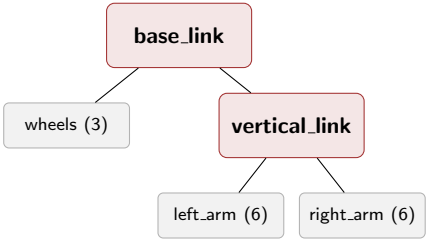


URDF Robot Model

Link/Joint Structure:

- `base_link` - Robot base
- `wheel1-3` - Omniwheels
- `vertical_move` - Lift
- `left_joint1..6` - Left arm
- `right_joint1..6` - Right arm

Total: 17 joints (3 + 1 + 6 + 6 + gripper)



Joint Names Configuration

```yaml
# config/joint_names_Aloha.yaml
controller_joint_names: [
    '',               # Empty (index 0)
    'wheel1', 'wheel2', 'wheel3',          # Wheels
    'vertical_move',                        # Lift
    'left_joint1', ..., 'left_joint6',     # Left arm
    'right_joint1', ..., 'right_joint6',   # Right arm
]
```

Joint Indexing:

- Index 0: Empty placeholder
- Index 1-3: Omniwheels
- Index 4: Lift mechanism
- Index 5-10: Left arm (6 DOF)
- Index 11-16: Right arm (6 DOF)

ROS1 Launch Files

display.launch (RViz):

```
<launch>
  <param name="robot_description"
    textfile="$(find Aloha)/urdf/Aloha.urdf"/>

  <node name="joint_state_publisher_gui"
    pkg="joint_state_publisher_gui"
    type="joint_state_publisher_gui"/>

  <node name="robot_state_publisher"
    pkg="robot_state_publisher"
    type="robot_state_publisher"/>

  <node name="rviz"
    pkg="rviz"
    type="rviz"
    args="-d $(find Aloha)/urdf.rviz"/>
</launch>
```

gazebo.launch (Physics):

```
<launch>
  <include file="$(find gazebo_ros)/
    launch/empty_world.launch"/>

  <node name="tf_footprint_base"
    pkg="tf"
    type="static_transform_publisher"
    args="0 0 0 0 0 0 base_link
          base_footprint 40"/>

  <node name="spawn_model"
    pkg="gazebo_ros"
    type="spawn_model"
    args="-file $(find Aloha)/urdf/
          Aloha.urdf -urdf -model Aloha"
    output="screen"/>
</launch>
```

ROS2 Launch File (Python)

```python
# display.launch.py
from ament_index_python.packages import get_package_share_directory
from launch import LaunchDescription
from launch_ros.actions import Node

def generate_launch_description():
    bringup_dir = get_package_share_directory('Aloha')
    urdf_path = os.path.join(bringup_dir, 'urdf', 'Aloha.urdf')

    with open(urdf_path, 'r') as f:
        robot_description = f.read()

    return LaunchDescription([
        Node(package='robot_state_publisher', executable='robot_state_publisher',
             parameters=[{'robot_description': robot_description}]),
        Node(package='joint_state_publisher', executable='joint_state_publisher'),
        Node(package='rviz2', executable='rviz2',
             arguments=['-d', rviz_config_path]),
    ])
```

Package Dependencies (package.xml)

```xml
<?xml version="1.0"?>
<package format="3">
  <name>Aloha</name>
  <version>0.1.0</version>
  <description>AlohaMini Robot</description>

  <buildtool_depend>ament_cmake</buildtool_depend>

  <exec_depend>joint_state_publisher</exec_depend>
  <exec_depend>joint_state_publisher_gui</exec_depend>
  <exec_depend>robot_state_publisher</exec_depend>
  <exec_depend>rviz2</exec_depend>
  <exec_depend>xacro</exec_depend>

  <export>
    <build_type>ament_cmake</build_type>
  </export>
</package>
```

RViz Visualization

Features:

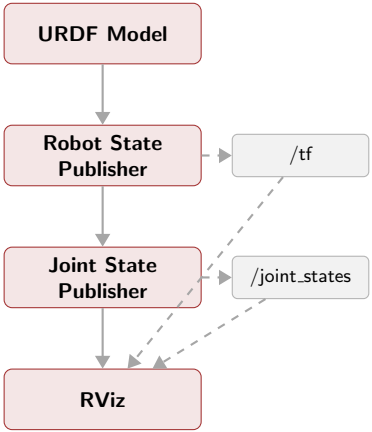
- Robot model visualization
- TF frame tree display
- Joint state publisher GUI
- Interactive joint control

Usage (ROS1):

- `roslaunch Aloha display.launch`

Usage (ROS2):

- `ros2 launch Aloha display.launch.py`



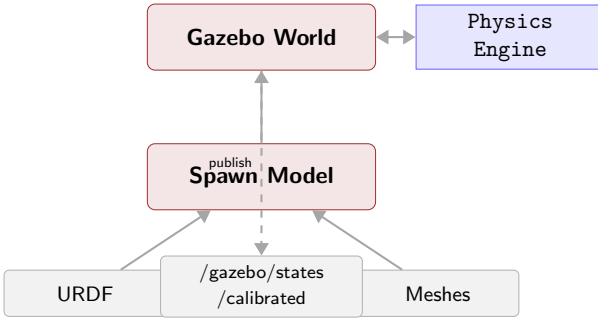
Gazebo Physics Simulation

Features:

- Full physics simulation
- Collision detection
- Inertia-based dynamics
- World interaction

Launch Components:

1. Empty world spawn
2. TF footprint publisher
3. Model spawner
4. Fake joint calibration



Usage:

- `roslaunch Aloha gazebo.launch`

Directory Structure

```
simulation/src/Aloha/
+-- config/
|   +-- joint_names_Aloha.yaml
+-- launch/
|   +-- display.launch       # ROS1
|   +-- display.launch.py    # ROS2
|   +-- gazebo.launch
+-- meshes/
|   +-- base_link.STL
|   +-- wheel*.STL
|   +-- left_joint*.STL
|   +-- right_joint*.STL
+-- urdf/
|   +-- Aloha.urdf
+-- package.xml
```

Mesh Files:

- STL format (binary)
- SolidWorks export
- Collision & visual

Build System:

- ament_cmake (ROS2)
- catkin (ROS1 compatible)

ROS Simulation Summary

Key Components:

- URDF robot model
- STL mesh files
- ROS1/ROS2 launch files
- Joint configuration

Visualization:

- RViz for visual inspection
- Joint state publisher GUI
- TF frame visualization

Physics Simulation:

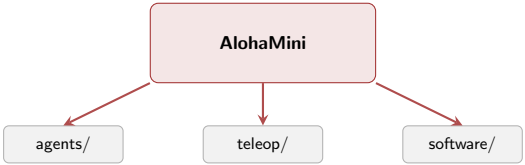
- Gazebo integration
- Inertia-based dynamics
- Collision meshes

Comparison with ManiSkill3:

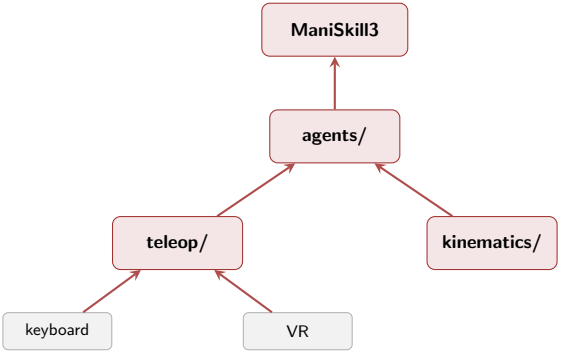
Feature	ROS	ManiSkill3
Physics	Gazebo	SAPIEN
GPU Accel	No	Yes
RL Support	Limited	Native
Visualization	RViz	Viewer

Directory Structure

```
AlohaMini/
+-- docs/           # Documentation
+-- hardware/       # CAD files
+-- maniskill/      # ManiSkill3
|   +-- agents/     # Robot agents
|   +-- teleop/     # Teleoperation
|   +-- assets/     # URDF/meshes
|   +-- examples/   # Demo scripts
+-- software/       # LeRobot
+-- simulation/     # ROS packages
```



Module Dependencies



Thank You

AlohaMini Codebase Documentation

https://github.com/AlohaMini